UNIT – II
Understanding Requirements and Requirements Modeling:Requirements engineering, Software requirement specification, Requirement analysis, Datamodeling, Flow oriented modeling, Behavioral modeling.

Understanding Requirements

Understanding the requirements of a problem is among the most difficulttasks that face a software engineer. When you first think about it, developinga clear understanding of requirements doesn't seem that hard. Afterall, doesn't the customer know what is required? Shouldn't the end users havea good understanding of the features and functions that will provide benefit?Surprisingly, in many instances the answer to these questions is "no." And even ifcustomers and end-users are explicit in their needs, those needs will changethroughout the project.

Requirements engineering

Requirement Engineering is the process of defining, documenting and maintaining the requirements. It is a process of gathering and defining service provided by the system. Requirements Engineering Process consists of the following main activities:

- Requirements elicitation
- Requirements specification
- Requirements verification and validation
- Requirements management

Requirements Elicitation:

It is related to the various ways used to gain knowledge about the project domain and requirements. The various sources of domain knowledge include customers, business manuals, the existing software of same type, standards and other stakeholders of the project.
The techniques used for requirements elicitation include interviews, brainstorming, task analysis, Delphi technique, prototyping, etc. Some of these are discussed here. Elicitation does not produce formal models of the requirements understood. Instead, it widens the domain knowledge of the analyst and thus helps in providing input to the next stage.

Requirements specification:
This activity is used to produce formal software requirement models. All the requirements including the functional as well as the non-functional requirements and the constraints are specified by these models in totality. During specification, more knowledge about the problem may be required which can again trigger the elicitation process.

The models used at this stage include ER diagrams, data flow diagrams(DFDs), function decomposition diagrams(FDDs), data dictionaries, etc.

Requirements verification and validation:

Verification: It refers to the set of tasks that ensures that the software correctly implements a specific function.
Validation: It refers to a different set of tasks that ensures that the software that has been built is traceable to customer requirements.
If requirements are not validated, errors in the requirement definitions would propagate to the successive stages resulting in a lot of modification and rework.
The main steps for this process include:

- The requirements should be consistent with all the other requirements i.e no two requirements should conflict with each other.
- The requirements should be complete in every sense.
- The requirements should be practically achievable.

Requirements management:

Requirement management is the process of analyzing, documenting, tracking, prioritizing and agreeing on the requirement and controlling the communication to relevant stakeholders. This stage takes care of the changing nature of requirements. It should be ensured that the SRS is as modifiable as possible so as to incorporate changes in requirements specified by the end users at later stages too. Being able to modify the software as per requirements in a systematic and controlled manner is an extremely important part of the requirements engineering process.

Establishing The Groundwork

In an ideal world, stakeholders and software engineers would collaborate on the same team. In such cases, requirements engineering is simply a matter of having meaningful conversation with team members who are well-known.

We go over the steps that must be taken to lay the groundwork for an understanding of software requirements-to get the project started in way that will keep it moving toward successful completion.

Identifying Stakeholders

Any person who benefits directly or indirectly from the system being developed is a stakeholder. Business operations managers, product managers, marketing people, internal and external customers, end-users, consultants, product engineers, software engineers, and support/maintenance engineers are the usual stakeholders.

Each stakeholder sees the system differently, gains different benefits when the system is successfully developed and faces different risks if the development effort fails.

Recognizing Multiple Viewpoints

Because there are so many different stakeholders, the system's requirements will be examined from various perspectives. Each of these stakeholders will contribute data to the requirements engineering process. As information is gathered from multiple viewpoints, emerging requirements may be inconsistent or contradictory. You should categorise all stakeholder information so that decision-makers can select an internally consistent set of system requirements for the system.

Working toward Collaboration

If there are five stakeholders involved in a software project, there may be five different opinions on the set of requirements. Customers must work together as well as with software engineering practitioners to create a successful system. A requirements engineer's job is to identify areas of commonality as well as areas of conflict or inconsistency. Collaboration does not always imply that requirements are set by a committee. In many cases, stakeholders collaborate by providing their perspective on requirements, but a strong "project champion" may ultimately decide on which requirements are accepted.

ELICITING REQUIREMENTS

Requirements elicitation (also called *requirements gathering*) combines elements ofproblem solving, elaboration, negotiation, and specification.

Collaborative Requirements Gathering

There have been numerous techniques of collaborative requirements collecting proposed.

- Meetings are held and attended by software engineers as well as other stakeholders.
- Preparation and participation rules are created.
- An agenda is proposed that is formal enough to cover all critical topics while being informal enough to allow for the free flow of ideas.
- The discussion is managed by a "facilitator."
- A "definition mechanism" (work sheets, flip charts, etc.) is used.

The purpose is to define the problem, offer solution elements, negotiate different approaches, and specify a preliminary set of solution requirements in an environment conducive to goal achievement. Each attendee is asked to make a list of objects that are part of the environment that surrounds the system, other objects that are to be produced by the system, and objects that are used by the system to perform its functions while reviewing the product request in the days leading up to the meeting.

The mini-specs are offered for discussion to all stakeholders. There are additions, deletions, and extra elaboration. In some circumstances, creating mini-specs will reveal new objects,

services, constraints, or performance requirements that will be added to the original listings. The team may raise an issue that cannot be handled during the meeting at any time. An issue list is kept so that these suggestions can be followed up on later.

Quality Function Deployment

Quality function deployment (QFD) is a quality management technique that translates customer expectations into technical software requirements. It is focused on maximising customer satisfaction from the software engineering process.

The QFD defines types categories of requirements:

Normal requirements : The objectives and goals that are mentioned for a product or system during customer meetings are called normal requirements. The customer is satisfied if these requirements are met. Requested forms of graphical displays, specified system functionalities, and defined levels of performance are examples of normal requirements.

Expected requirements : These requirements are inherent in the product or system and may be so basic that the client does not express them explicitly. Their absence will be a major source of dissatisfaction. Expected needs include ease of human/machine communication, general operating correctness and reliability, and software installation simplicity.

Exciting requirements : These characteristics go above and beyond the customer's expectations and are absolutely delightful when they are present. For example, software for a new mobile phone has typical functionality as well as a set of unexpected capabilities like multitouch screen, visual voice mail, etc. that thrill every product user.

Usage Scenarios : As requirements are acquired, an overall vision of system functions and features emerges. However, moving into more technical software engineering activities is difficult unless you understand how these functions and features will be used by different types of end-users. To do this, engineers and users can collaborate to establish a set of scenarios that identify a thread of usage for the system to be built.

Elicitation Work Products:-

The work products generated as a result of requirements elicitation will differ according to the

size of the system or product to be constructed.

The work products for the majority of systems include

- A need and feasibility statement.
- A bounded statement of the system's or product's scope.
- A list of clients, users, and other stakeholders who took part in the requirements elicitation process.
- A description of the technological environment of the system.

- A list of objectives, as well as the domain restrictions that apply to each of them.
- A set of usage scenarios that provide information about how the system or product is used under various operational conditions.
- Any prototypes created to help describe requirements more precisely.

Each of these work products is assessed by everyone who took part in the requirements

elicitation process.

Software Requirement Specification (SRS)

The production of the requirements stage of the software development process is **Software Requirements Specifications (SRS)** (also called a **requirements document**). This report lays a foundation for software engineering activities and is constructing when entire requirements are elicited and analyzed. **SRS** is a formal report, which acts as a representation of software that enables the customers to review whether it (SRS) is according to their requirements. Also, it comprises user requirements for a system as well as detailed specifications of the system requirements.

The SRS is a specification for a specific software product, program, or set of applications that perform particular functions in a specific environment. It serves several goals depending on who is writing it. First, the SRS could be written by the client of a system. Second, the SRS could be written by a developer of the system. The two methods create entirely various situations and establish different purposes for the document altogether. The first case, SRS, is used to define the needs and expectation of the users. The second case, SRS, is written for various purposes and serves as a contract document between customer and developer.

Characteristics of good SRS

Following are the features of a good SRS document:

1. Correctness: User review is used to provide the accuracy of requirements stated in the SRS. SRS is said to be perfect if it covers all the needs that are truly expected from the system.

2. Completeness: The SRS is complete if, and only if, it includes the following elements:

(1). All essential requirements, whether relating to functionality, performance, design, constraints, attributes, or external interfaces.

(2). Definition of their responses of the software to all realizable classes of input data in all available categories of situations.

(3). Full labels and references to all figures, tables, and diagrams in the SRS and definitions of all terms and units of measure.

3. Consistency: The SRS is consistent if, and only if, no subset of individual requirements described in its conflict. There are three types of possible conflict in the SRS:

(1). The specified characteristics of real-world objects may conflicts. For example,

(a) The format of an output report may be described in one requirement as tabular but in another as textual.

(b) One condition may state that all lights shall be green while another states that all lights shall be blue.

(2). There may be a reasonable or temporal conflict between the two specified actions. For example,

(a) One requirement may determine that the program will add two inputs, and another may determine that the program will multiply them.

(b) One condition may state that "A" must always follow "B," while other requires that "A and B" co-occurs.

(3). Two or more requirements may define the same real-world object but use different terms for that object. For example, a program's request for user input may be called a "prompt" in one requirement's and a "cue" in another. The use of standard terminology and descriptions promotes consistency.

4. Unambiguousness: SRS is unambiguous when every fixed requirement has only one interpretation. This suggests that each element is uniquely interpreted. In case there is a method used with multiple definitions, the requirements report should determine the implications in the SRS so that it is clear and simple to understand.

Ranking for importance and stability: The SRS is ranked for importance and stability if each requirement in it has an identifier to indicate either the significance or stability of that particular requirement.

Typically, all requirements are not equally important. Some prerequisites may be essential, especially for life-critical applications, while others may be desirable. Each element should be identified to make these differences clear and explicit. Another way to rank requirements is to distinguish classes of items as essential, conditional, and optional.

6. Modifiability: SRS should be made as modifiable as likely and should be capable of quickly obtain changes to the system to some extent. Modifications should be perfectly indexed and cross-referenced.

7. Verifiability: SRS is correct when the specified requirements can be verified with a cost-effective system to check whether the final software meets those requirements. The requirements are verified with the help of reviews.

8. Traceability: The SRS is traceable if the origin of each of the requirements is clear and if it facilitates the referencing of each condition in future development or enhancement documentation.

There are two types of Traceability:

1. Backward Traceability: This depends upon each requirement explicitly referencing its source in earlier documents.

2. Forward Traceability: This depends upon each element in the SRS having a unique name or reference number.

The forward traceability of the SRS is especially crucial when the software product enters the operation and maintenance phase. As code and design document is modified, it is necessary to be able to ascertain the complete set of requirements that may be concerned by those modifications.

9. Design Independence: There should be an option to select from multiple design alternatives for the final system. More specifically, the SRS should not contain any implementation details.

10. Testability: An SRS should be written in such a method that it is simple to generate test cases and test plans from the report.

11. Understandable by the customer: An end user may be an expert in his/her explicit domain but might not be trained in computer science. Hence, the purpose of formal notations and symbols should be avoided too as much extent as possible. The language should be kept simple and clear.

12. The right level of abstraction: If the SRS is written for the requirements stage, the details should be explained explicitly. Whereas,for a feasibility study, fewer analysis can be used. Hence, the level of abstraction modifies according to the objective of the SRS

. Properties of a good SRS document

The essential properties of a good SRS document are the following:

Concise: The SRS report should be concise and at the same time, unambiguous, consistent, and complete. Verbose and irrelevant descriptions decrease readability and also increase error possibilities.

Structured: It should be well-structured. A well-structured document is simple to understand and modify. In practice, the SRS document undergoes several revisions to cope up with the user requirements. Often, user requirements evolve over a period of time. Therefore, to make the modifications to the SRS document easy, it is vital to make the report well-structured.

Black-box view: It should only define what the system should do and refrain from stating how to do these. This means that the SRS document should define the external behavior of the system and not discuss the implementation issues. The SRS report should view the system to be developed as a black box and should define the externally visible behavior of the system. For this reason, the SRS report is also known as the black-box specification of a system.

Conceptual integrity: It should show conceptual integrity so that the reader can merely understand it. Response to undesired events: It should characterize acceptable responses to unwanted events. These are called system response to exceptional conditions.

Verifiable: All requirements of the system, as documented in the SRS document, should be correct. This means that it should be possible to decide whether or not requirements have been met in an implementation.

Prototyping Model in Software Engineering

The **prototyping model** is applied when detailed information related to input and output requirements of the system is not available. In this model, it is assumed that all the requirements may not be known at the start of the development of the system. It is usually used when a system does not exist or in case of a large and complex system where there is no manual process to determine the requirements. This model allows the users to interact and experiment with a working model of the system known as **prototype.** The prototype gives the user an actual feel of the system.

At any stage, if the user is not satisfied with the prototype, it can be discarded and an entirely new system can be developed.

Generally, prototype can be prepared by the approaches listed below.

• By creating main user interfaces without any substantial coding so that users can get a feel of how the actual system will appear.

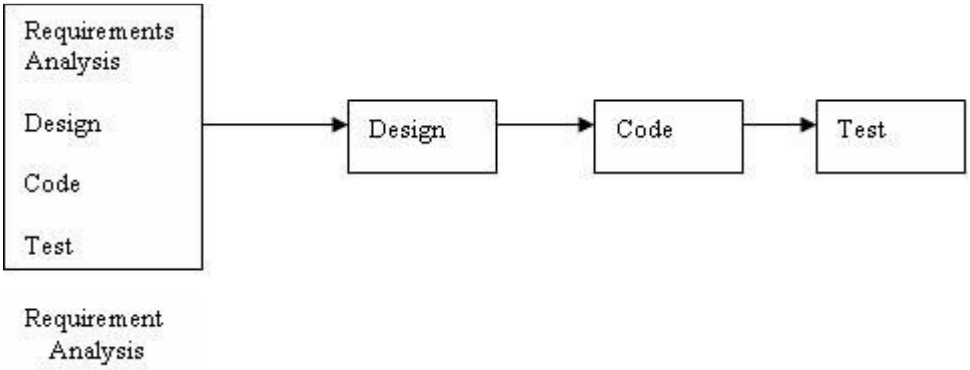
• By abbreviating a version of the system that will perform limited subsets of functions.

• By using system components to illustrate the functions that will be included in the system to be developed .

Using the prototype, the client can get an actual feel of the system. So, this case of model is beneficial in the case when requirements cannot be freezed initially.

This prototype is developed based on the currently known requirements. Development of the prototype obviously undergoes design, coding, and testing, but each of these phases is not done very formally or thoroughly.

By using this prototype, the client can get an actual feel of the system, because the interactions with the prototype can enable the client to better understand the requirements of the desired system.



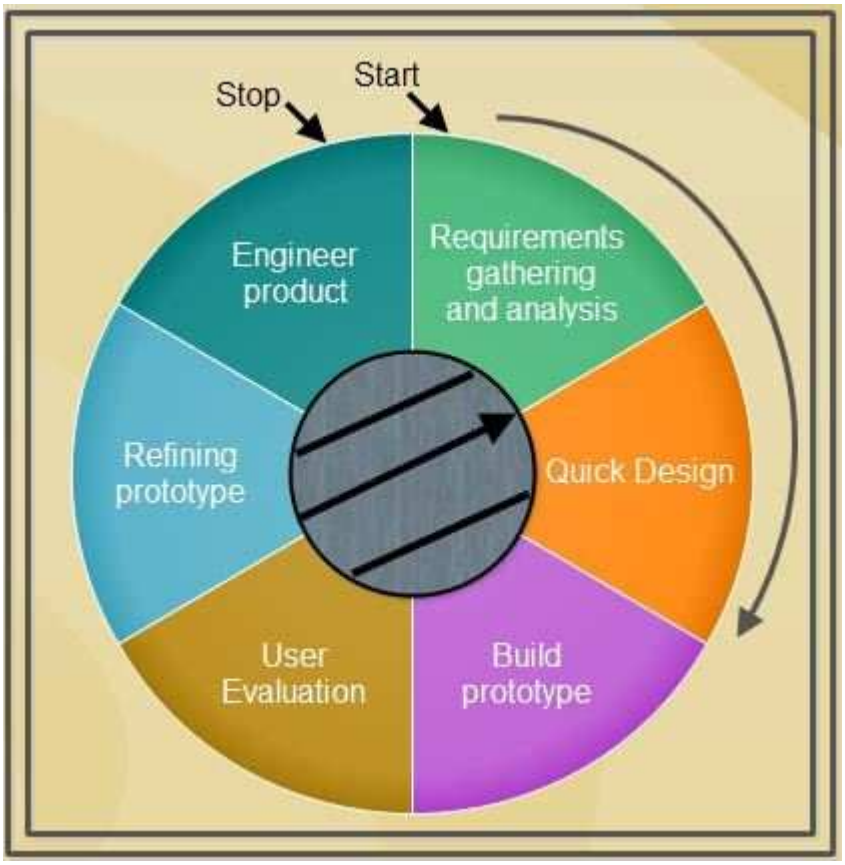
Prototyping is an attractive idea for complicated and large systems for which there is no manual process or existing system to help determine the requirements. Risks associated with the projects are being reduced through the use of prototyping. The development of the prototype typically starts when the preliminary version of the requirements specification document has been developed.

At this stage, there is a reasonable understanding of the system and its needs are unclear or likely to change. After the prototype has been developed, the end users and clients are given an opportunity to use the prototype.

They provide feedback to the developers regarding the prototype: what is correct, what needs to be modified, what is missing, what is not needed, etc. Based on the feedback, the prototype is modified to incorporate some of the suggested changes that can be done easily, and then the users and the clients are again allowed to use the system.

This cycle repeats until, in the judgment of the prototypes and analyst. Based on the feedback, the initial requirements are modified to produce that final requirements specification, which is then used to develop the production quality system.

Figure Illustrates the steps carried out in the prototyping model. These steps are listed below.



1. Requirements gathering and analysis: A prototyping model begins with requirements analysis and the requirements of the system are defined in detail. The user is interviewed in order to know the requirements of the system.

2. Quick design: When requirements are known, a preliminary design or quick design for the system is created. It is not a detailed design and includes only the important aspects of the system, which gives an idea of the system to the user. A quick design helps in developing the prototype.

3. Build prototype: Information gathered from quick design is modified to form the first prototype, which represents the working model of the required system.

4. User evaluation: Next, the proposed system is presented to the user for thorough evaluation of the prototype to recognize its strengths and weaknesses such as what is to be added or removed. Comments and suggestions are collected from the users and provided to the developer.

5. Refining prototype: Once the user evaluates the prototype and if he is not satisfied, the current prototype is refined according to the requirements. That is, a new prototype is developed with the additional information provided by the user. The new prototype is evaluated just like the previous prototype. This process continues until all the requirements

specified by the user are met. Once the user is satisfied with the developed prototype, a final system is developed on the basis of the final prototype.

6. Engineer product: Once the requirements are completely met, the user accepts the final prototype. The final system is evaluated thoroughly followed by the routine maintenance on regular basis for preventing large-scale failures and minimizing downtime.

Various advantages and disadvantages associated with the prototyping model are listed in Table.

Table *Advantages* and Disadvantages of Prototyping Model

Advantages	**Disadvantages**
1. Provides a working model to the user early in the process, enabling early assessment and increasing user's confidence. 2. The developer gains experience and insight by developing a prototype there by resulting in better implementation of requirements. 3. The prototyping model serves to clarify requirements, which are not clear, hence reducing ambiguity and improving communication between the developers and users. 4. There is a great involvement of users in software development. Hence, the requirements of the users are met to the greatest extent. 5. Helps in reducing risks associated with the software.	1. If the user is not satisfied by the developed prototype, then a new prototype is developed. This process goes on until a perfect prototype is developed. Thus, this model is time consuming and expensive. 2. The developer loses focus of the real purpose of prototype and hence, may compromise with the quality of the software. For example, developers may use some inefficient algorithms or inappropriate programming languages while developing the prototype. 3. Prototyping can lead to false expectations. For example, a situation may be created where the user believes that the development of the system is finished when it is not. 4. The primary goal of prototyping is speedy development, thus, the system design can suffer as it is developed in series without considering integration of all other components.

SYSTEM MODELING

System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system. It is about representing a system using some kind of graphical notation, which is now almost always based on notations in the **Unified Modeling Language** (**UML**). Models help the analyst to understand the functionality of the system; they are used to communicate with customers.

Models can explain the system from **different perspectives**:

- An **external** perspective, where you model the context or environment of the system.
- An **interaction** perspective, where you model the interactions between a system and its environment, or between the components of a system.
- A **structural** perspective, where you model the organization of a system or the structure of the data that is processed by the system.
- A **behavioral** perspective, where you model the dynamic behavior of the system and how it responds to events.

Five types of UML diagrams that are the most useful for system modeling:

- **Activity** diagrams, which show the activities involved in a process or in data processing.
- **Use case** diagrams, which show the interactions between a system and its environment.
- **Sequence** diagrams, which show interactions between actors and the system and between system components.
- **Class** diagrams, which show the object classes in the system and the associations between these classes.
- **State** diagrams, which show how the system reacts to internal and external events.

Models of both new and existing system are used during **requirements engineering**. Models of the **existing systems** help clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses. These then lead to requirements for the new system. Models of the **new system** are used during requirements engineering to help explain the

proposed requirements to other system stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation.

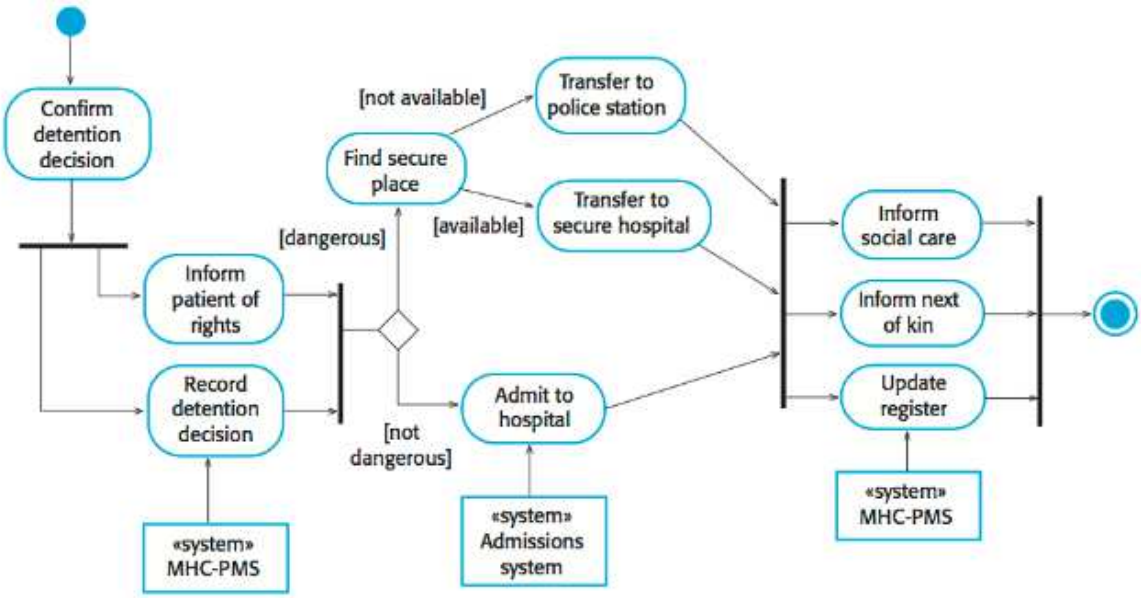
CONTEXT AND PROCESS MODELS

Context models are used to illustrate the operational context of a system - they show what lies outside the system boundaries. Social and organizational concerns may affect the decision on where to position system boundaries. Architectural models show the system and its relationship with other systems.

System boundaries are established to define what is inside and what is outside the system. They show other systems that are used or depend on the system being developed. The position of the system boundary has a profound effect on the system requirements. Defining a system boundary is a political judgment since there may be pressures to develop system boundaries that increase/decrease the influence or workload of different parts of an organization.

Context models simply show the other systems in the environment, not how the system being developed is used in that environment. **Process models** reveal how the system being developed is used in broader business processes. UML activity diagrams may be used to define business process models.

The example below shows a UML **activity diagram** describing the process of involuntary detention and the role of MHC-PMS (mental healthcare patient management system) in it.

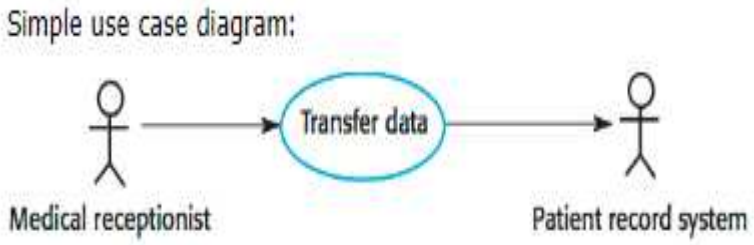


INTERACTION MODELS

Types of interactions that can be represented in a model:

- Modeling **user interaction** is important as it helps to identify user requirements.
- Modeling **system-to-system interaction** highlights the communication problems that may arise.
- Modeling **component interaction** helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.

Use cases were developed originally to support requirements elicitation and now incorporated into the UML. Each use case represents a discrete task that involves external interaction with a system. Actors in a use case may be people or other systems. Use cases can be represented using a UML use case diagram and in a more detailed textual/tabular format.

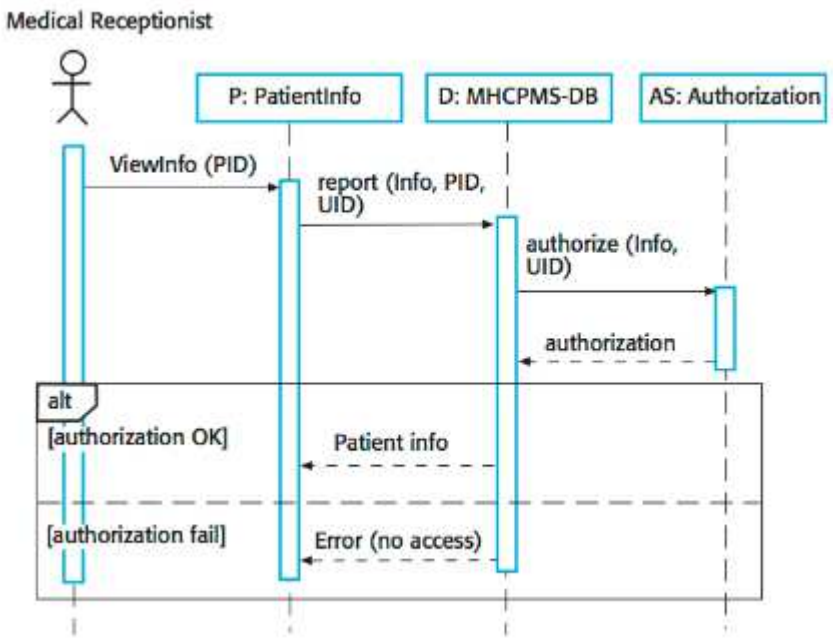


Use case description in a tabular format:

Use case title	Transfer data
Description	A receptionist may transfer data from the MHC-PMS to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Actor(s)	Medical receptionist, patient records system (PRS)
Preconditions	Patient data has been collected (personal information, treatment summary); The receptionist must have appropriate security permissions to access the patient information and the PRS.

Postconditions	PRS has been updated
Main success scenario	1. Receptionist selects the "Transfer data" option from the menu. 2. PRS verifies the security credentials of the receptionist. 3. Data is transferred. 4. PRS has been updated.
Extensions	2a. The receptionist does not have the necessary security credentials. 2a.1. An error message is displayed. 2a.2. The receptionist backs out of the use case.

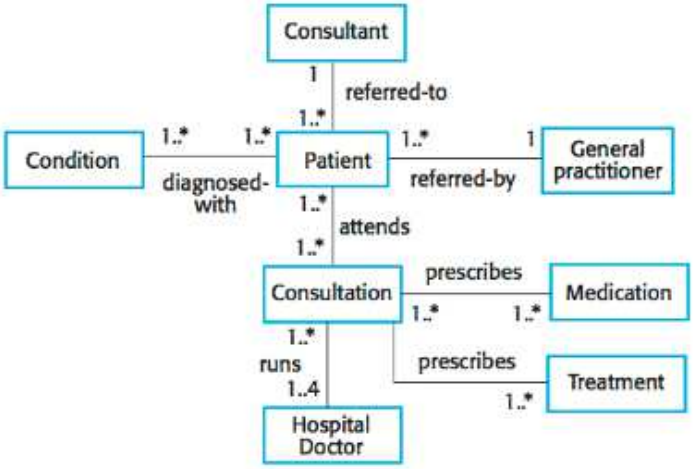
UML **sequence diagrams** are used to model the interactions between the actors and the objects within a system. A sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance. The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these. Interactions between objects are indicated by annotated arrows.



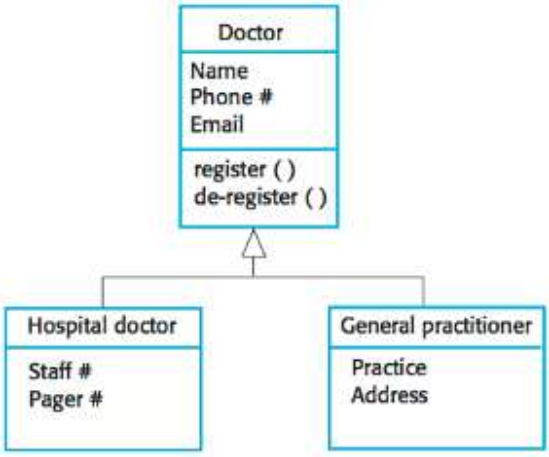
STRUCTURAL MODELS

Structural models of software display the organization of a system in terms of the components that make up that system and their relationships. Structural models may be **static** models, which show the structure of the system design, or **dynamic** models, which show the organization of the system when it is executing. You create structural models of a system when you are discussing and designing the system architecture.

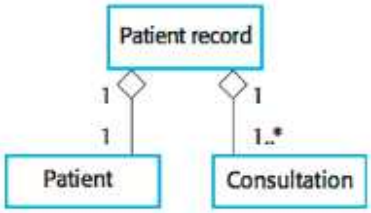
UML **class diagrams** are used when developing an object-oriented system model to show the classes in a system and the associations between these classes. An object class can be thought of as a general definition of one kind of system object. An association is a link between classes that indicates that there is some relationship between these classes. When you are developing models during the early stages of the software engineering process, objects represent something in the real world, such as a patient, a prescription, doctor, etc.



Generalization is an everyday technique that we use to manage complexity. In modeling systems, it is often useful to examine the classes in a system to see if there is scope for generalization. In object-oriented languages, such as Java, generalization is implemented using the class **inheritance** mechanisms built into the language. In a generalization, the attributes and operations associated with higher-level classes are also associated with the lower-level classes. The lower-level classes are subclasses inherit the attributes and operations from their superclasses. These lower-level classes then add more specific attributes and operations.



An **aggregation** model shows how classes that are collections are composed of other classes. Aggregation models are similar to the part-of relationship in semantic data models.

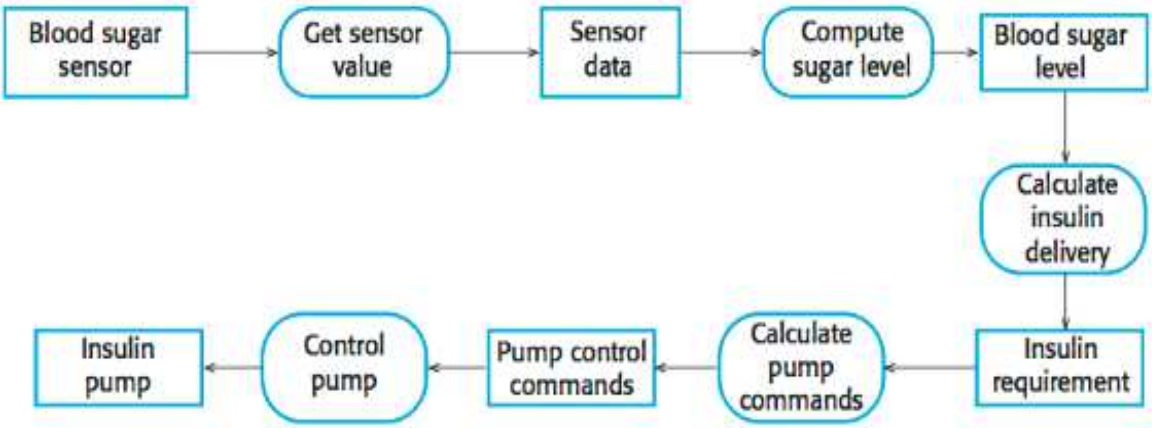


BEHAVIORAL MODELS

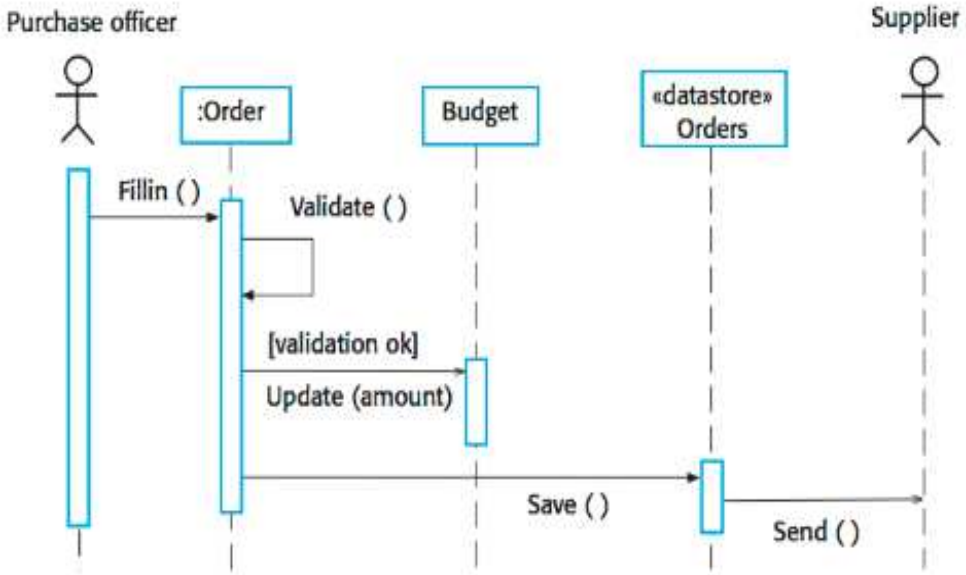
Behavioral models are models of the dynamic behavior of a system as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus from its environment. Two types of stimuli:

- Some **data** arrives that has to be processed by the system.
- Some **event** happens that triggers system processing. Events may have associated data, although this is not always the case.

Many business systems are data-processing systems that are primarily driven by data. They are controlled by the data input to the system, with relatively little external event processing. **Data-driven models** show the sequence of actions involved in processing input data and generating an associated output. They are particularly useful during the analysis of requirements as they can be used to show end-to-end processing in a system. Data-driven models can be created using UML **activity diagrams**:

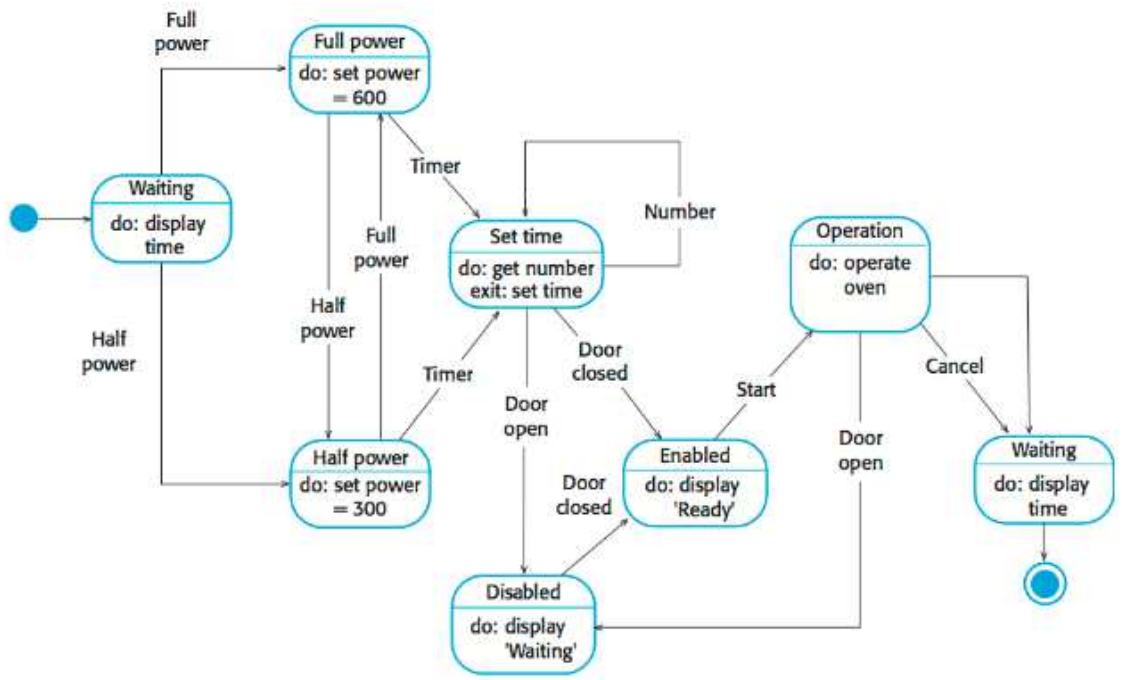


Data-driven models can also be created using UML **sequence diagrams**:



REAL-TIME SYSTEMS

Real-time systems are often event-driven, with minimal data processing. For example, a landline phone switching system responds to events such as 'receiver off hook' by generating a dial tone. **Event-driven models** shows how a system responds to external and internal events. It is based on the assumption that a system has a finite number of states and that events (stimuli) may cause a transition from one state to another. Event-driven models can be created using UML **state diagrams**:



Key Points:-

● A model is an abstract system view. Complementary types of model provide different system information.

● Context models show the position of a system in its environment with other systems and processes.

● Data flow models may be used to model the data processing in a system.

● State machine models model the system's behaviour in response to internal or external events.

● Semantic data models describe the logical structure of data which is imported to or exported by the systems.

- Object models describe logical system entities, their classification and aggregation.
- Sequence models show the interactions between actors and the system objects that they use.
- Structured methods provide a framework for developing system models.

DATA MODELING VS. REQUIREMENTS MODELING

To understand data modeling and how it contributes to software engineering, it is best to take a step back and examine how data modeling fits into requirements modeling.

Requirements modeling in software engineering is part of analysis and design. It's the planning stage of developing a software application. Requirements modeling focuses on the 'what', not the 'how'. In other words, requirements modeling identifies what requirements the application must meet in order to be successful.

Requirements modeling includes many sub-stages, one of them being **data modeling**. Generally speaking, requirements modeling will begin with scenario-based modeling, which results in creating a use case. A **use case**, simply put, is a primary example of how the software application will be used and what it is expected to do. Once a use case exists, one of the stages that follow will be data modeling.

Data Objects, Attributes, and Relationships

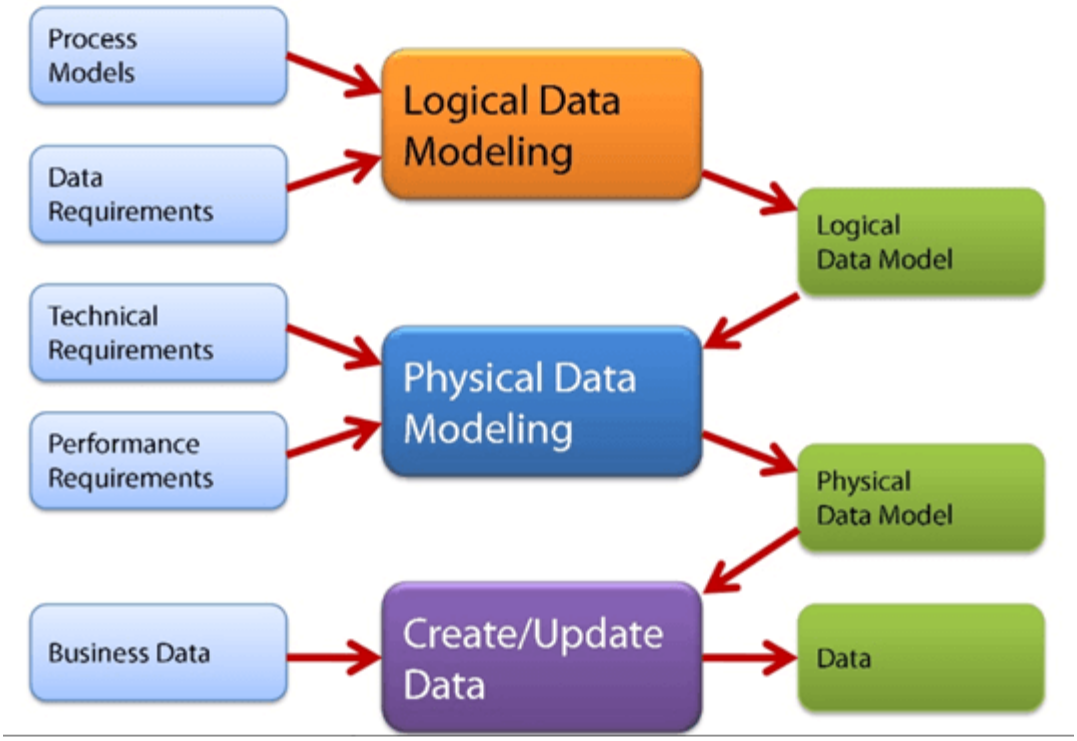
Data modeling, sometimes also called information modeling, is the process of visually representing what data the application or system will use, and how it will flow. The resulting diagram or other visual representation is meant to be designed in a way that is as easy to understand as possible. The fundamental elements that a data model needs to include and describe are the **data objects**, more frequently called **'entities'**, the **attributes** of those objects/entities, and the **relationships** between the objects/entities.

Entities, attributes and relationships are reviewed in greater detail in the example use case that follows. We won't be going into diagrams in this lesson, however it is worth noting that several methods can be used to create the diagrams in data modeling. The two most widely used are **UML** (Unified Modeling Language), and **ERD** (Entity-Relationship Diagrams).

CONCEPTUAL, LOGICAL, AND PHYSICAL MODELING

1. **Conceptual Data Model:** This Data Model defines **WHAT** the system contains. This model is typically created by Business stakeholders and Data Architects. The purpose is to organize, scope and define business concepts and rules.

2. **Logical Data Model:** Defines **HOW** the system should be implemented regardless of the DBMS. This model is typically created by Data Architects and Business Analysts. The purpose is to developed technical map of rules and data structures.
3. **Physical Data Model**: This Data Model describes **HOW** the system will be implemented using a specific DBMS system. This model is typically created by DBA and developers. The purpose is actual implementation of the database.



FUNCTIONAL MODELING AND INFORMATION FLOW MODELING

In the **Functional Model**, software converts information. and to accomplish this, it must perform at least three common tasks- input, processing and output. When functional models of an application are created, the software engineer emphasizes problem specific tasks. The functional model begins with a single reference level model (i.e., be manufactured). In a series of iterations, more and more functional detail is given, until all system functionality is fully represented.

Information is converted because it flows from a computer-based system. The system takes input in various forms; Hardware, software, and human elements are applied to replace it; And produces in various forms. The transformation (s) or function may be composed of a single logical comparison, a complex numerical method, or a rule- the invention approach of an expert system. The output can light an LED or provide a 200 page report. Instead, we can create a model or flow model for any computer- based system, regardless of size and complexity.

Structural analysis started as an **Information Flow Modeling** technique. A computer-based system can be modeled as an information transform function as shown in figure.

A rectangle represents an external unit. That is, a system element, such as a hardware, a person or another system that provides information for transformation by the software or receives information provided by the software. A circle is used to represent a process or transform or a function that is applied to data and changes it in some way. An arrow is used to represent one or more data items.

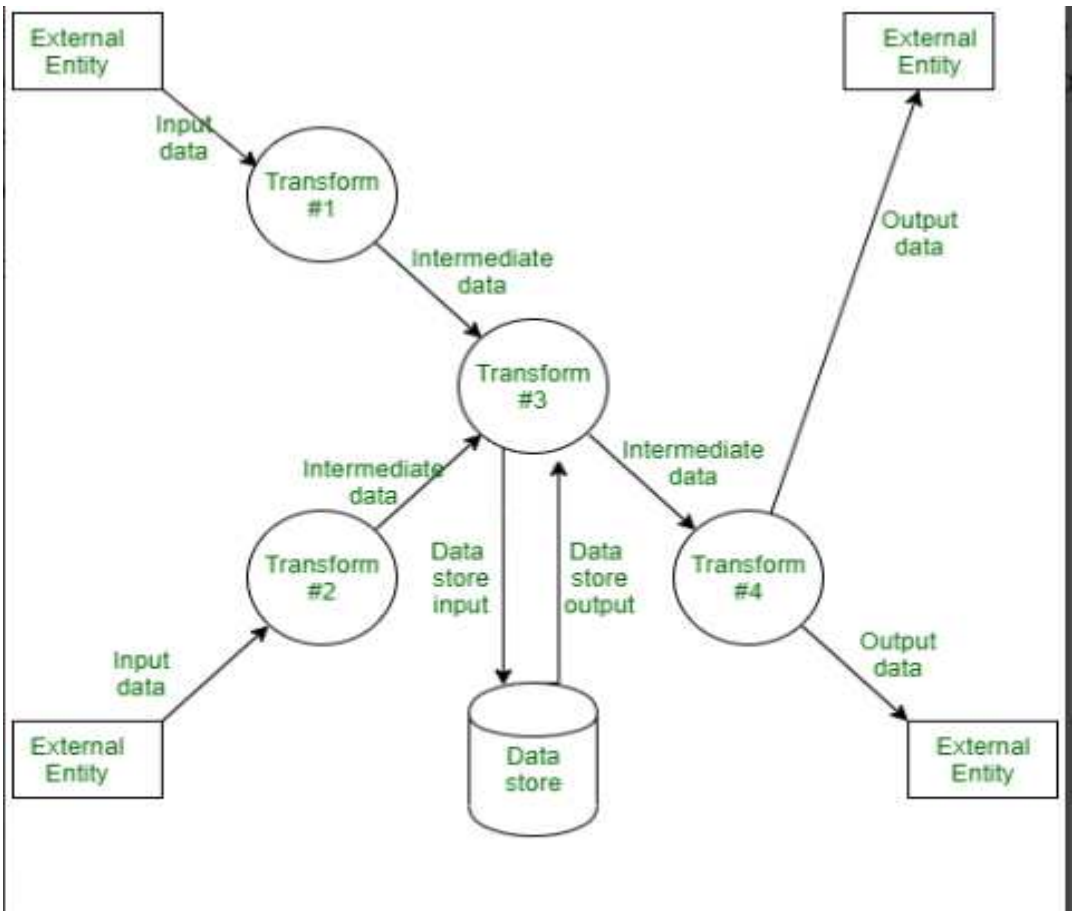


Figure - Information-flow Model

All arrows should be labeled in a DFD. The double line is used to represent data store. There may be implicit procedure or sequence in the diagram but explicit logical details are generally delayed until software design.

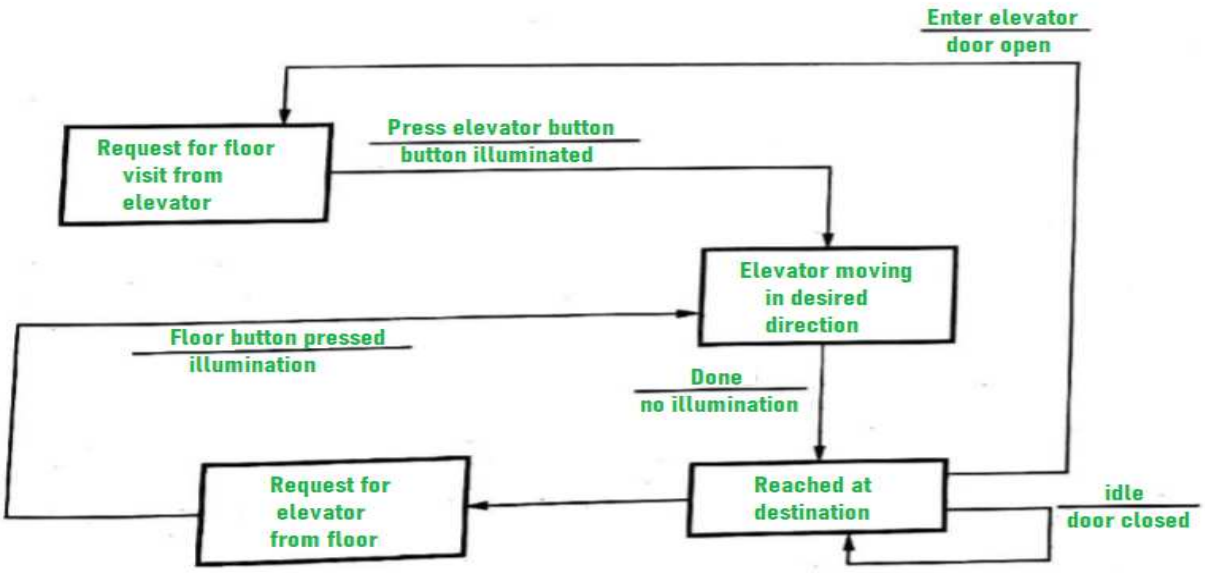
Behavioral Model

Behavioral Model is specially designed to make us understand behavior and factors that influence behavior of a System. Behavior of a system is explained and represented with the help of a diagram. This diagram is known as State Transition Diagram. It is a collection of states and events. It usually describes overall states that a system can have and events which are responsible for a change in state of a system.
So, on some occurrence of a particular event, an action is taken and what action needs to be taken is represented by State Transition Diagram.

Example :
Consider an Elevator. This elevator is for n number of floors and has n number of buttons one for each floor.

Elevator's working can be explained as follows :

1. **Elevator buttons** are type of set of buttons which is there on elevator. For reaching a particular floor you want to visit, "elevator buttons" for that particular floor is pressed. Pressing, will cause illumination and elevator will start moving towards that particular floor for which you pressed "elevator buttons". As soon as elevator reaches that particular floor, illumination gets canceled.
2. **Floor buttons** are another type of set of buttons on elevator. If a person is on a particular floor and he wants to go on another floor, then elevator button for that floor is pressed. Then, process will be same as given above. Pressing, will cause illumination and elevator to start moving, and when it reaches on desired floor, illumination gets canceled.
3. When there is no request for elevator, it remains closed on current floor.



STATE TRANSITION DIAGRAM

Advantages :

- Behavior and working of a system can easily be understood without any effort.
- Results are more accurate by using this model.
- This model requires less cost for development as cost of resources can be minimal.
- It focuses on behavior of a system rather than theories.

Disadvantages :

- This model does not have any theory, so trainee is not able to fully understand basic principle and major concept of modeling.
- This modeling cannot be fully automated.
- Sometimes, it's not easy to understand overall result.
- Does not achieve maximum productivity due to some technical issues or any errors.